Universidad de Huelva

4º Curso de Grado
en Ingeniería Informática
Especialidad en Computación

Área de Ciencias de la
Computación e Inteligencia Artificial
Departamento de
Tecnologías de la Información

PRÁCTICA 2 (Versión 2026, 1.0)

Algoritmos de Búsqueda Multiarranque: GRASP, GRASP Extendido, ILS y VNS

Objetivos

El objetivo de esta práctica es estudiar el funcionamiento de los Algoritmos de Búsqueda Multiarranque: Procedimiento de Búsqueda Voraz Aleatorio Adaptativo (GRASP), Búsqueda Local Iterativa (ILS) y Búsqueda Basada en Entornos Cambiantes (VNS). Implemente estos algoritmos, para resolver el problema de optimización de balanceo de una red de bicicletas en una ciudad visto en la práctica 1. El comportamiento de los algoritmos implementados deberá compararse entre sí y, al menos, también con las técnicas Greedy y Búsqueda Local del Primer Mejor de la Práctica 1. El análisis de los resultados influirá decisivamente en la calificación de la práctica

Enunciado de la práctica

Si no se especifica lo contrario asumir los mismos elementos de la práctica en este enunciado para cada algoritmo.

- *Algoritmo de Búsqueda Local*: Utilice la Búsqueda Local del Primer Mejor Vecino con el intercambio como operador de generación de vecinos (algoritmo 2-opt) propuesto asimismo en la práctica anterior.

Función Objetivo

El propósito del modelo es **minimizar la distancia total generada en el sistema y la entropía al inicio**,

Los kilómetros totales seran varias veces superior a la entropía en un caso pequeño y al revés en un caso con muchas estaciones, por lo que si se opta por sumar ambas cantidades, una solución *mediocre* (porque dos soluciones con una de las partes mala y la otra buena) darian

valores similares, no guiando bien al algoritmo, es tener una suma ponderada con un coeficiente *Alpha.*

$$Fobj = Kms +/- Alpha * Entropia$$

Para obtener mejores valores estudiar el uso de otras formulas, ratio kms/entropía o exponenciales, etc… proponer la mejor según el resultado.

GRASP

Algoritmo

El algoritmo GRASP constará de dos componentes: construcción de soluciones *Greedy* probabilísticas, y optimización de estas mediante el algoritmo de búsqueda local. El algoritmo *Greedy* probabilístico es el siguiente:

El mecanismo GRASP seleccionará iterativamente la siguiente estación a visitar formando una Lista de Candidatos Restringida (RCL) de tamaño 3 con las opciones no visitadas de "mayor potencial" (basándose en heurística de distancia y entropía a decidir por el alumno). Luego, seleccionará aleatoriamente una de la lista RCL con probabilidad ponderada.

Una vez generada cada solución probabilística, se aplicará la Búsqueda Local del primer mejor.

Valores de los Parámetros: Se generarán 10 soluciones Greedy probabilísticas para cada ejecución.

ILS

El algoritmo ILS consistirá en generar una solución aleatoria inicial y aplicar el algoritmo de Búsqueda Local Primer Mejor. Una vez obtenida la solución optimizada, se estudiará si es mejor que la mejor solución encontrada hasta el momento y se realizará una mutación fuerte sobre la mejor de las dos, volviendo a aplicar el algoritmo de búsqueda local sobre esta solución mutada. Este proceso se repite 10 veces, devolviéndose la mejor solución encontrada en todo el proceso. Por tanto, se sigue el *Criterio del Mejor* como criterio de aceptación.

El *operador de mutación fuerte* se implementará de la siguiente forma: *Sublista Aleatoria de Tamaño Fijo*: este proceso consiste en generar aleatoriamente dos posiciones que determinen una sublista de tamaño *4*, analizar las asignaciones efectuadas entre ambas y reasignarlas aleatoriamente. Se considerarán subsistas cíclicas y, por tanto, la posición inicial de la sublista no siempre será inferior a la posición final. Si está más cerca del final que el tamaño, se terminará cogiendo valores del principio de la solución candidata (vector ciclico)

VNS

Algoritmo

El algoritmo VNS que consideraremos se compone de los siguientes pasos:

1. Generar la solución actual aleatoria S_{act} y hacer $k = 1$, $bl = 0$
2. Si ($k > k_{max}$) hacer $k = 1$.
3. Generar una solución vecina (S_{vec}) de S_{act} con el operador de generación de vecino para el valor del parámetro k, $S_{vec} = N_k(S_{act})$.
4. Aplicar la Búsqueda Local sobre la solución S_{vec}, obteniendo S'. Hacer $bl = bl + 1$.
5. Si S' es mejor que S_{act}, hacer $S_{act} = S'$ y $k = 1$. Si no, hacer $k = k + 1$.
6. Si ($bl < bl_{max}$) volver a 2. Si no, devolver S_{act} y terminar.

El operador de generación de vecino hará uso de nuevo de la *Sublista Aleatoria de Tamaño Fijo* anteriormente descrita para el algoritmo ILS.

Valores de los Parámetros

El algoritmo de búsqueda local se aplicará diez veces ($bl_{max} = 10$). Se trabajará con $k_{max} = 4$, es decir, cuatro entornos diferentes. El valor de s (tamaño de la sublista) en el operador de generación de vecino definirá el tamaño del movimiento según el valor de k. Irá aumentándose de la siguiente forma:

- $k = 1$: Se aplica un tamaño de $s = n/6$.
- $k = 2$: Se aplica un tamaño de $s = n/5$.
- $k = 3$: Se aplica un tamaño de $s = n/4$.
- $k = 4$: Se aplica un tamaño de $s = n/3$.

Los resultados que deben aportarse

El comportamiento de los algoritmos implementados deberá compararse entre sí y con las técnicas *Greedy* y Búsqueda Local del primer mejor vecino de la Práctica 1.

Finalmente, se construirá una tabla global de resultados con la estructura mostrada en la Tabla 1.2 para número de evaluaciones, Función objetivo ,kms y entropia .

	Caso 1				Caso 2				Caso 3			
Modelo	#Ev	F OBJ	Kms	Entr	#Ev	F OBJ	Kms	Entr	#Ev	F OBJ	Kms	Entr
Greedy	1				1				1			
P.Mejor												
GRASP												
ILS												
VNS												

Tabla 1.1. Resultados Globales

Elegir el mejor resultado de las ejecuciones realizadas.

Medidas de Estudio y Evaluación de Algoritmos

Problema del Balanceo de 17 Estaciones de Bicicletas
(Análisis de Caja Negra y Caja Blanca para GRASP, VNS e ILS)

La evaluación del rendimiento de algoritmos probabilísticos y metaheurísticas se analiza tradicionalmente bajo dos perspectivas complementarias: el enfoque de caja negra, que evalúa el rendimiento final, y el enfoque de caja blanca, que analiza el comportamiento interno del proceso de búsqueda. A continuación, se proponen medidas y gráficas

1. Medidas de Evaluación de Caja Negra (Black-Box)

Las medidas de caja negra ignoran cómo el algoritmo llega a la solución y se centran en la calidad, costo y estabilidad del resultado final frente a múltiples ejecuciones.

1.1. Robustez Matemática (Desviación y Coeficiente de Variación)

Dado que estos algoritmos tienen factores aleatorios, una sola ejecución no es representativa. La industria utiliza la Media y la Desviación Típica (σ) de múltiples ejecuciones (ej. 5 o 10 ejecuciones independientes de GRASP/ILS/VNS con distintas semillas).

- Coeficiente de Variación :

$$CV = (\sigma / Media) * 100.$$

Un CV muy bajo (< 5%) indica que el algoritmo es muy robusto y estable, siempre entregando calidades similares sin importar las semillas iniciales.

1.2. Error Relativo Porcentual (RPD / RE)

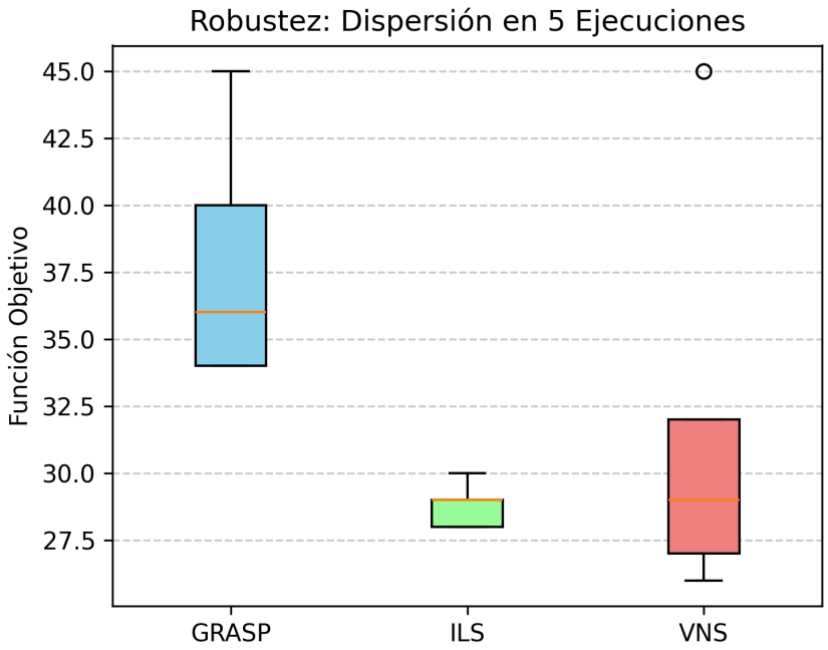
Para medir qué tan lejos se queda un algoritmo del mejor resultado posible conocido.

- RPD = ((FObj_obtenida - FObj_mejor_conocido) / FObj_mejor_conocido) * 100.

Permite comparar la calidad de ILS vs VNS de manera estandarizada independientemente del tamaño del dataset. Usar el mejor resultado de la Práctica 1 como Fobj mejor_conocido.

1.3. Gráfica de Caja Negra: Boxplot (Diagrama de Caja y Bigotes)

G un Boxplot que compare GRASP, VNS e ILS en el eje X, y la Función Objetivo final en el eje Y.
Interpretación: Una caja pequeña y compacta indica alta robustez y poca dispersión. Una caja baja y cercana al cero indica mejor calidad. Cualquier punto fuera de los bigotes (outliers) alertará de ejecuciones que quedaron atrapadas en pésimos óptimos locales.



2. Medidas de Evaluación de Caja Blanca (White-Box)

Estas métricas estudian la trayectoria y el comportamiento interno del algoritmo para verificar si explora correctamente o si se estanca prematuramente. Es ideal para comparar el mecanismo de "ruido" de ILS, los entornos de VNS y el multi-arranque de GRASP.

2.1. Tasa de Diversificación Estructural (Overlap o Distancia de Hamming)

Mide cuánto varía el recorrido del camión entre las soluciones finales de distintas ejecuciones. Si la distancia de Hamming de la ruta R es nula entre las 5 ejecuciones de GRASP, significa que la fase Greedy está siempre creando la misma semilla inicial (falta diversificación).

Ejemplo

Supongamos que analizamos las soluciones finales (rutas) arrojadas por 3 ejecuciones distintas del algoritmo GRASP sobre el Caso 1

Ruta de Ejecución 1: [0, 3, 5, 2, 1, 9, 8...]
Ruta de Ejecución 2: [0, 3, 5, 2, 7, 4, 10...]
Ruta de Ejecución 3: [0, 3, 5, 2, 6, 12, 11...]

Al comparar estas rutas mediante la Distancia de Hamming (posición por posición), observamos que el "prefijo" de las estaciones [3, 5, 2] es idéntico en las tres ejecuciones. Esto arroja una tasa de "Overlap" (solapamiento) extremadamente alta en las fases tempranas de la ruta. Interpretación para el alumno: El parámetro del tamaño de la Lista Restringida de Candidatos (RCL) en la fase Greedy del GRASP es demasiado pequeño o determinista. El algoritmo está arrancando siempre hacia las estaciones 3, 5 y 2 porque su heurística de entropía es tan fuerte allí, que asfixia el factor aleatorio. Para solucionarlo e inyectar diversidad, se debería aumentar el parámetro l (tamaño de RCL), permitiendo que rutas tempranas comiencen visitando otras estaciones necesitadas como la 4 o la 7.

2.2. Profundidad del Estancamiento (Stagnation Depth)

Se refiere al recuento del número de evaluaciones consecutivas donde el algoritmo no ha mejorado su Función Objetivo.

- En ILS: ¿Cuántas iteraciones de Búsqueda Local se gastan en promedio antes de tener que aplicar la mutación de la sublista?
- En VNS: ¿Con qué frecuencia el algoritmo se ve obligado a cambiar al entorno más grande (E.g. k=5) porque no encuentra mejoras en los pequeños?

Ejemplo
Si analizamos los registros continuos de una única ejecución y miramos la métrica de estancamiento (evaluaciones perdidas en las que no bajaban ni los kilómetros recorridos ni la entropía):

Ejemplo en VNS:

observamos un registro : [Entorno 1: 0 mejoras], [Entorno 2: 0 mejoras], [Entorno 3: 0 mejoras], [Entorno 4: 1 Mejora de -12kms].

Esto significa que el algoritmo acumula un estancamiento muy profundo en bucles cortos y rutinariamente tiene que escalar hasta k=4 (aplicando la mutación de mayor tamaño) para mover sustancialmente la estructura de la ruta.
Interpretación: Los entornos pequeños de VNS (variar solo 1 o 2 estaciones adyacentes) no sirven en este tipo de redes geográficas de bicicletas porque un mínimo valle local es muy amplio geográficamente.
Se puede concluir que rediseñar los vecindarios o saltarse directamente k=1 y k=2 le ahorraría miles de evaluaciones inútiles (reduciendo el coste de CPU).

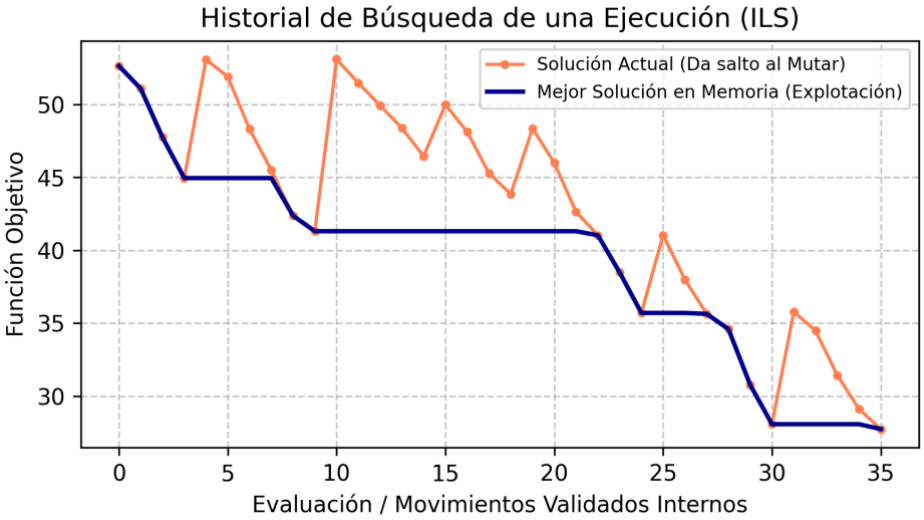
Ejemplo en ILS:

El log indica que la Búsqueda Local hace en promedio 2,500 iteraciones revisando todos los intercambios de pares posibles antes de confirmar que está en un óptimo local y activar el mecanismo de Mutación de Sublista.

Interpretación para el alumno: Si la Búsqueda Local Primera o Mejor requiere 2,500 llamadas a la función de coste por ciclo antes de permitir la mutación, el ILS pasará el 98% del tiempo arrastrándose en el fondo del valle local en vez de explorar el mapa. La profundidad de estancamiento es demasiado larga.
El alumno debería optar por una "Búsqueda Local de Primer Mejor" con cortes más rápidos o truncamiento para permitir que ILS aplique su mutación de distorsión con mayor frecuencia.

2.3. Gráfica de Caja Blanca: Evolución Dual Exploración/Explotación

Gráfica a generar: Una gráfica de línea que no solo muestre la curva monótona de la "Mejor Función Objetivo hasta el momento", sino que sobreponga la "Función Objetivo de la Solución Actual" en cada iteración o mutación.
Interpretación: En ILS y VNS veremos "picos" bruscos hacia arriba (empeoramiento) cada vez que se aplica la mutación o cambio de entorno (exploración), seguido de rápidas caídas hacia abajo (explotación) en la Búsqueda Local. El alumno deberá analizar si los picos son suficientemente altos para escapar del óptimo local anterior.



3. Ejemplo Práctico con los Datos del Caso 1

El camión (L=20) inicia con 7 bicis
. Ejemplo de la Estación 4: Bicis=8, Capacidad=21. Al 50% ideal serían ~10 bicis, el camión debe depositar 2 bicis.

Ejemplo de Análisis de las Gráficas Anteriores:

Supongamos quese ejecutan los tres algoritmos y obtiene los siguientes costes agrupados de 5 ejecuciones (Kms + Entropía ponderada):

- GRASP: [34, 45, 36, 34, 40]
- ILS: [28, 29, 29, 30, 28]
- VNS: [26, 32, 27, 45, 29]

Medida de Caja Negra (Robustez/Calidad mostrada en el Boxplot):

Al observar los datos representados en el Boxplot, ILS tiene la Media más consistente (~28.8) y la menor desviación (caja verde compacta), demostrando ser altamente robusto. VNS logró el mejor valor absoluto (26), pero es inestable y de alta varianza ya que su bigote superior se estira hacia el salto del 45.
Medida de Caja Blanca (Mostrada en la Gráfica de Trayectoria de ILS):
La línea de trayectoria naranja nos permite observar el interior de una de las ejecuciones. Cada pico ascendente es el momento exacto en el que el ILS usó el parámetro de la sublista aleatoria (s=n/4) para romper un mínimo local (exploración), provocando un empeoramiento súbito. Este alejamiento era inmediatamente absorbido y optimizado drásticamente mediante

los movimientos de la Búsqueda Local (línea descendente en diagonal), ayudando a descender el récord de Kms+Entropía marcado por la línea azul inferior, rebasando las cuotas previas y sacándolo del estancamiento

Fecha y Método de Entrega:

Límite, día 8 de Mayo 10:00am Defensa según horario preestablecido ese día.
Documentación: Análisis de resultados y Código fuente.

Fichero PDF y código fuente comprimidos con zip o rar se subirán a la plataforma *moodle* de EV. ***Téngase en cuenta que el análisis, cuenta tanto como las implementaciones y resultados***

Permanezca atento a posibles versiones mejoradas de este documento.

Anexo: Propuestas de Función Objetivo Avanzada

La suma ponderada clásica (Kms + Alpha × Entropía) suele sufrir problemas de escala porque el algoritmo encuentra atajos: prefiere arruinar el balanceo de una estación (sacrificando un poco de entropía) si eso le ahorra cruzar toda la ciudad (reduciendo drásticamente los kilómetros).
Si el objetivo primario en la industria es garantizar el balanceo (lograr la Entropía Máxima del 50%) y el objetivo secundario es que la ruta sea lo más corta posible, la mejor forma de guiar la búsqueda es usar una Fórmula de Penalización Exponencial. La entropía máxima en este problema es de 16 (ya que partimos de 0, siendo el resto de las estaciones la parte que hay que optimizar.

1. La Fórmula de Escalado Exponencial (Priorización de Inventario)

Variables del sistema:

- Kms: Kilómetros recorridos por el camión (Queremos minimizarlo).
- E_{max}: Entropía máxima ideal del sistema (Si hay 16 estaciones, $E_{max} = 16$).
- E: Entropía alcanzada por la solución actual.
- β: Factor de rigor (Típicamente un valor de escalado entre 1 y 3).

La función a minimizar sería:

$$F_{obj} = Kms \cdot e^{\beta \cdot (E_{max} - E)}$$

¿Por qué funciona esta fórmula?

Fase 1 (Presión abrumadora sobre el balanceo):
La diferencia entra la entropía máxima y la actual representa el "déficit de entropía" (qué tan mal balanceado está el sistema frente al ideal). Al estar en el exponente, si el déficit es alto, el multiplicador literalmente explota. Una ruta cortísima pero que no balancea las bicis tendrá un coste artificial de millones. El algoritmo se verá forzado a ignorar los kilómetros en sus primeras etapas y enfocarse desesperadamente en reducir ese exponente.

Fase 2 (Sintonía fina de la ruta):
A medida que el algoritmo logra balancear el sistema, la entropía alcanzada se acerca al ideal. El déficit tiende a cero. Matemáticamente, cualquier número elevado a cero es 1. Cuando el sistema está casi perfectamente equilibrado, la penalización exponencial desaparece pacíficamente y la fórmula muta de forma natural a la optimización de distancia pura. En esta etapa, el algoritmo usará sus exploraciones locales única y exclusivamente para hacer la ruta más corta sin estropear el balanceo.

2. Alternativa: Exponencial de Insatisfacción Relativa

Otra alternativa derivada que también fuerza un objetivo antes que otro es elevar la variable menos importante a la "potencia del grado de insatisfacción" de la variable más importante.

$$F_{obj} = Kms^{(1 + E_{max} - E / E_{max})}$$

- **Si la entropía es malísima:** El déficit es total (1.0). La fórmula evalúa los Kms elevados al cuadrado. Las distancias largas se penalizan cuadráticamente, causando rechazo sistemático a soluciones malas en el modelo.
- **Si la entropía es perfecta:** El déficit es 0.0. La fórmula evalúa los Kms elevados a 1. Convirtiendo la métrica de nuevo a la normalidad lineal y permitiendo afinar los viajes.